Design and Implementation of a Line Following and Obstacle Avoidance Robot

Oleg Kelner*, Petr Lavrenov*, Turja Barua*
*Systems Engineering and Prototyping, BSc Electronic Engineering
Hochschule Hamm-Lippstadt, Lippstadt, Germany
{oleg.kelner, petr.lavrenov, turja.barua}@stud.hshl.de

***Abstract*—This paper presents the design, development, and implementation of an autonomous mobile robot capable of both line following and obstacle avoidance. The project was conducted as part of a university prototyping course, focusing on integrating hardware components such as infrared sensors, ultrasonic modules, and servo motors with an Arduino-based control system. The robot's behavior is governed by a state machine model to ensure reliable navigation and quick response to environmental changes. Supporting tools like 3D modeling for hardware mounting, Tinkercad for simulation, and SysML for system-level design were used throughout the development process. The final prototype demonstrates effective performance in real-time scenarios, providing a practical understanding of embedded systems, control logic, and systems engineering principles.**

***Index Terms*—Autonomous robot, Line following, Obstacle avoidance, Arduino, Embedded systems, Infrared sensor, Ultrasonic sensor.**

I. Introduction

This project focuses on building a simple autonomous robot that can follow a line and avoid obstacles using basic sensors. The robot is controlled by an Arduino Uno and uses infrared sensors to detect the line and an ultrasonic sensor to measure distance from obstacles. We used a step-by-step design process that included system modeling, 3D printing of parts, and simulation in Tinkercad. A state machine was used in the software to control how the robot switches between line-following and obstacle-avoidance modes. The goal was to learn how to combine hardware and software to create a working prototype.

II. System Design and Engineering

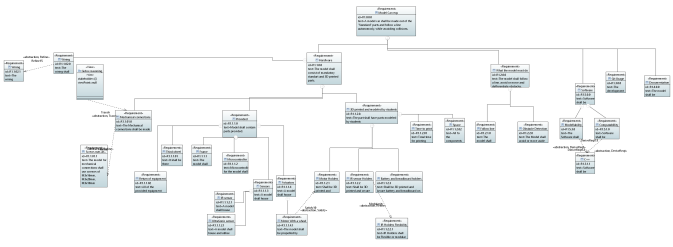

Fig. 1. ProfessorsRequirementDiagram (full page view in Appendix A)

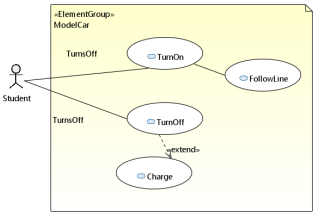

Fig. 2. Use Case Diagram(Student)

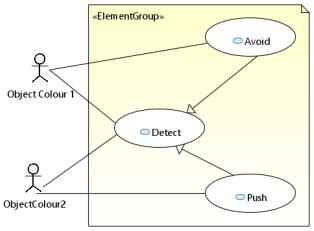

Fig. 3. Use Case Diagram(Object-Oriented)

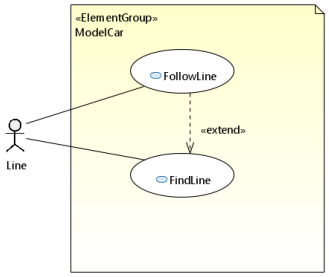

Fig. 4. LineUseCase

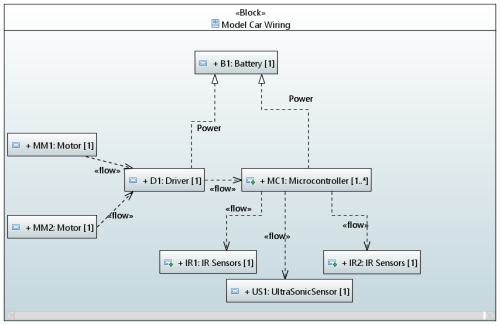

Fig. 5. ModelCarInternalWiringBlockDiagram

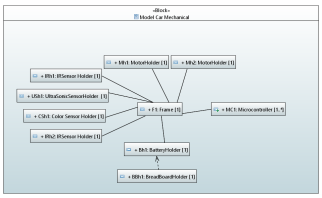

Fig. 6. Mechanical Internal Block Diagram

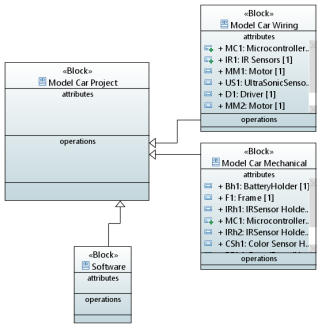

Fig. 7. Model Car Block Definition Diagram

III. Prototyping and Design

A. Electronic Components

The robot was built using a combination of basic electronic components. An Arduino Uno served as the central controller, processing sensor inputs and controlling the motors. Infrared (IR) sensors were used to detect the line path, while an ultrasonic sensor was employed for obstacle detection. A pair of DC motors provided movement, and the motor driver controlled their direction and speed. A breadboard and jumper wires were used to connect all components during testing and prototyping.

B. Virtual and Physical Prototypes

All components were mounted onto a custom laser-cut wooden chassis. The Arduino board, breadboard, and motor driver were securely placed, and wiring was organized for stable operation. The ultrasonic sensor is mounted on a servo motor to allow directional scanning. The wheels are driven by DC motors placed at the rear, and all parts are powered via a shared voltage rail.

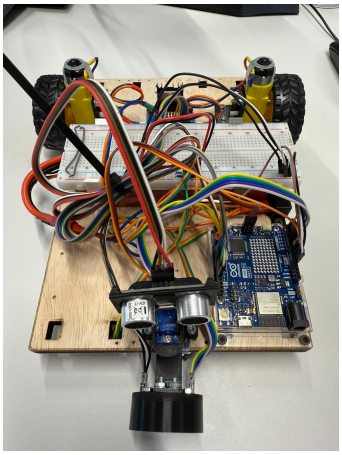

Fig. 8. Robot Assembly

C. Hardware Assembly

All components were securely mounted on a custom-designed chassis and connected according to the system schematic. The IR sensors, ultrasonic sensor, motor driver, and Arduino Uno were all arranged for optimal functionality and accessibility. To better suit the physical layout and wiring constraints, the entire robot assembly was reversed, meaning the front-facing sensors and drive motors were reoriented to face the opposite direction. This design change ensured improved cable management, balanced weight distribution, and more efficient component interaction during operation.

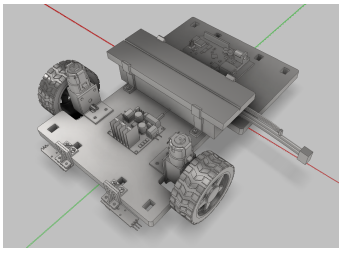

Fig. 9. Assembly

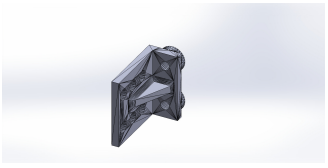

Fig. 10. IR Sensor Holder 1

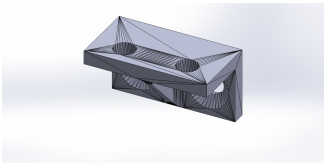

Fig. 11. IR Sensor Holder 2

IV. Circuit Design

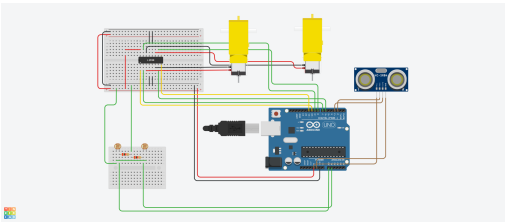

Fig. 12. TinkerCad Simulation

Circuit Simulation Layout:

The TinkerCad simulation shows the practical wiring of the robot's components on a breadboard. The Arduino Uno is connected to an motor driver IC for motor control, while IR sensors detect line position and the ultrasonic sensor handles distance measurement. All connections are routed through the breadboard with clear separation between power and control lines for easy testing and debugging.

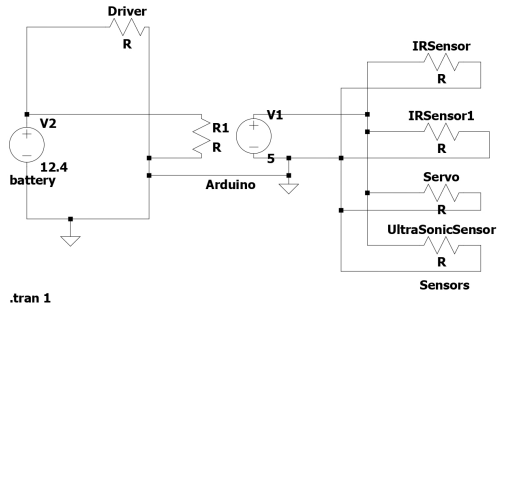

Fig. 13. Power Wiring

Power Supply Configuration

The power wiring diagram models the voltage distribution of the system. A 12.4V battery supplies power to the motor driver, while the Arduino Uno regulates and provides a 5V output to power the sensors and other components. The IR sensors, ultrasonic sensor, and servo motor are connected in parallel to the Arduino's 5V and GND pins. Proper wiring and current distribution ensure safe and stable operation of all modules.

V. Software Implementation

A. Initialization of Sensors and Actuators

Pins for sensors were defined and initialized in the 'setup()' function. Below is a snippet:

Listing 1. Sensor Initialization Code

```
void setup() {
  pinMode(2, INPUT);   // IR sensor
  pinMode(9, OUTPUT);  // Motor
  Serial.begin(9600);
}
```

B. Movement Control Algorithms

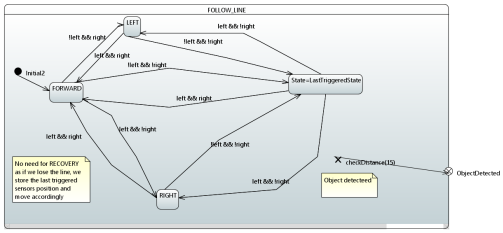

Fig. 14. FollowLineStateMachine

1. Follow Line State Machine: This state machine guides the robot along a line using infrared sensors. It starts in the `GoForward` state. If the left sensor detects the line, it transitions to `GoLeft`; if the right sensor detects the line, it moves to `GoRight`. If neither sensor sees the line, it returns to `GoForward`. The state `LoseLine` represents a failure to detect the path entirely.

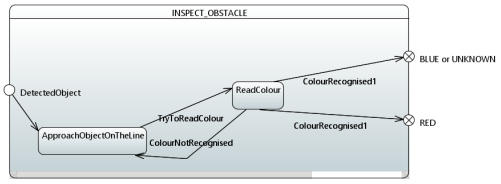

Fig. 15. Recognise Object State Machine

2. Recognise Object State Machine: This logic is activated when the robot detects an object. It first enters the `ApproachObjectOnTheLine` state, then attempts to read the color in `ReadColour`. Based on the result, it transitions to either `Colour1` or `Colour2`, or loops back if the color is not recognized. This helps determine how the robot should react to different objects.

C. Behavioral Decision Logic

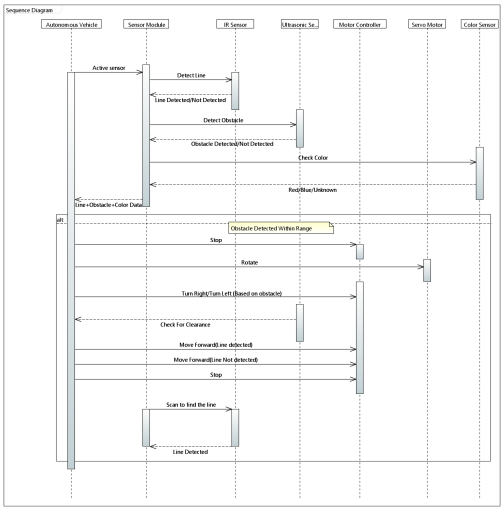

Fig. 16. SequenceDiagram

1. Sequence Diagram: This diagram illustrates how components interact over time. The IR sensor detects line positions, while the ultrasonic sensor checks for obstacles. Both send data to the microcontroller, which processes it to control the motors. If an obstacle is detected, the system initiates an avoidance routine and then resumes line following, ensuring continuous decision-making.

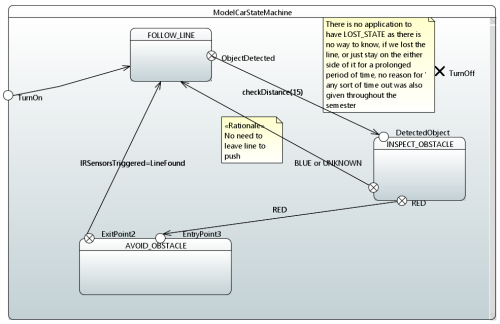

Fig. 17. ModelCarStateMachine

2. Model Car State Machine: The state machine shows high-level decision-making. The robot starts in the `Follow Line` state. Upon detecting an object, it transitions to `RecogniseObject`. Based on the detected color, it either continues pushing (Color1) or avoids the object (Color2). After avoiding, it returns to line following. This logic enables adaptive behavior based on environmental inputs.

VI. Git Usage and Collaboration

Project development was tracked using GitHub.

Team Member	Number of Commits
Oleg Kelner	77
Petr Lavrenov	57
Turja Barua	11
Total	**145**

TABLE I
GitHub Commits by Team Members

VII. Key Achievements

- The robot successfully follows a predefined path using IR sensors.
- It can accurately differentiate between colors using an onboard color sensor.
- It is capable of detecting and avoiding obstacles in real-time using an ultrasonic sensor.

VIII. Acknowledgment

We would like to express our sincere gratitude to our course instructor and project supervisor for their continuous guidance and valuable feedback throughout the development of this project. We are also thankful to Hochschule Hamm-Lippstadt for providing the tools, resources, and a collaborative environment that made this work possible. Lastly, we appreciate the teamwork and commitment of all group members in bringing this project to completion.

References

[1] Arduino Documentation. https://www.arduino.cc/en/Guide
[2] Tinkercad Simulation. https://www.tinkercad.com
[3] Uppaal Model Checker. https://www.uppaal.org/

Declaration of Originality

We hereby declare that this report is our own work and that we have acknowledged all sources used.

APPENDIX A: PROFESSORSREQUIREMENTDIAGRAM